Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	1	0	0	1	1	0	0

Figure 6.22 Sequences of input and output signals.

Steps->

- State diagram
- State table
- State assigned table
- K-map
- Circuit

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

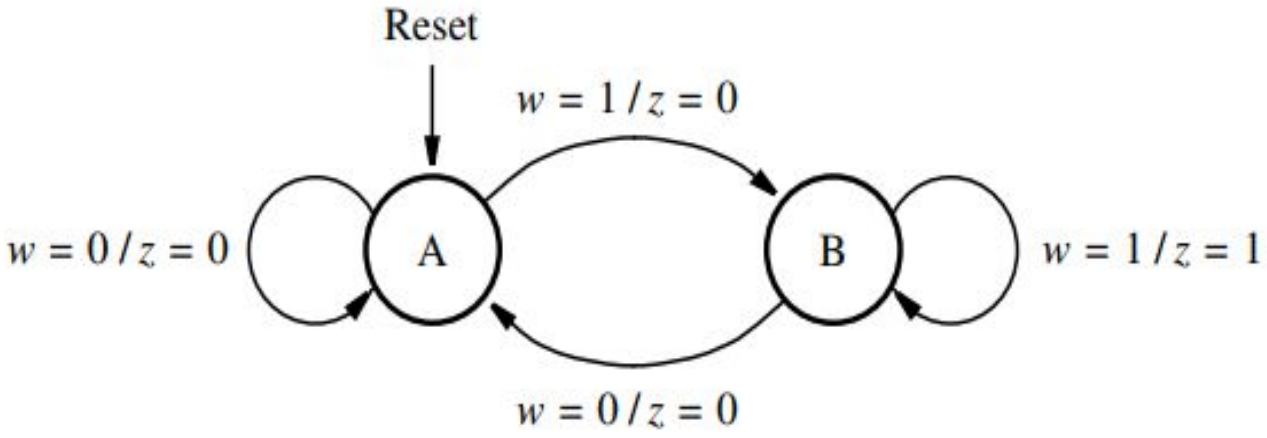


Figure 6.23 State diagram of an FSM that realizes the task in Figure 6.22.

Steps->

- **State diagram**
- State table
- State assigned table
- K-map
- Circuit

Example 2: A Simple Input Pattern ('11' Overlapping Sequence) Detection Circuit (Mealy Type)

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

Figure 6.24 State table for the FSM in Figure 6.23.

Steps->

- State diagram
- **State table**
- **State assigned table**
- K-map
- Circuit

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

Figure 6.25 State-assigned table for the FSM in Figure 6.24.

Note that,

- Y = f(w,y)
- z = f(w,y)

Example 2: A Simple Input Pattern (‘11’ Overlapping Sequence) Detection Circuit (Mealy Type)

Y =>

y \ w	0	1
0	0	1
1	0	1

Y = w

z =>

y \ w	0	1
0	0	0
1	0	1

z = wy

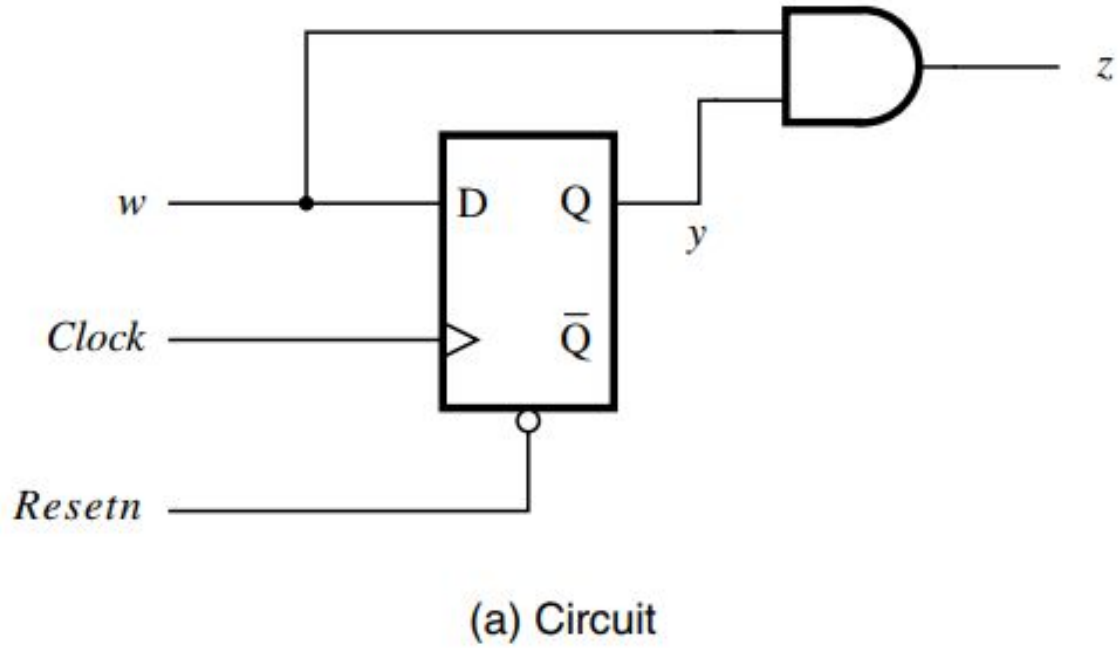
Steps->

- State diagram
- State table
- State assigned table
- **K-map** (Y and z)
- Circuit

Note that,

- Y = f(w,y)
- z = f(w,y)

Example 2: A Simple Input Pattern (‘11’ Overlapping Sequence) Detection Circuit (Mealy Type)



(a) Circuit

Steps->

- State diagram
- State table
- State assigned table
- K-map
- **Circuit**

Encoding Schemes (State Assignment)

Consider a state assigned table below:

	Present state $y_2 y_1$	Next state $w=0$ $Y_2 Y_1$	Next state $w=1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

Schemes->

- **Binary encoding**
- Gray encoding
- One-hot encoding

Encoding Schemes (State Assignment)

Consider another state assigned table below:

	Present state $y_2 y_1$	Next state $w=0$ $Y_2 Y_1$	Next state $w=1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	*dd*	*dd*	*d*

Schemes->

- Binary encoding
- **Gray encoding**
- One-hot encoding

Decimal Number	4 bit Binary Number ABCD	4 bit Gray Code $G_1G_2G_3G_4$
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1
10	1 0 1 0	1 1 1 1
11	1 0 1 1	1 1 1 0
12	1 1 0 0	1 0 1 0
13	1 1 0 1	1 0 1 1
14	1 1 1 0	1 0 0 1
15	1 1 1 1	1 0 0 0

Encoding Schemes (State Assignment)

Consider another state assigned table below:

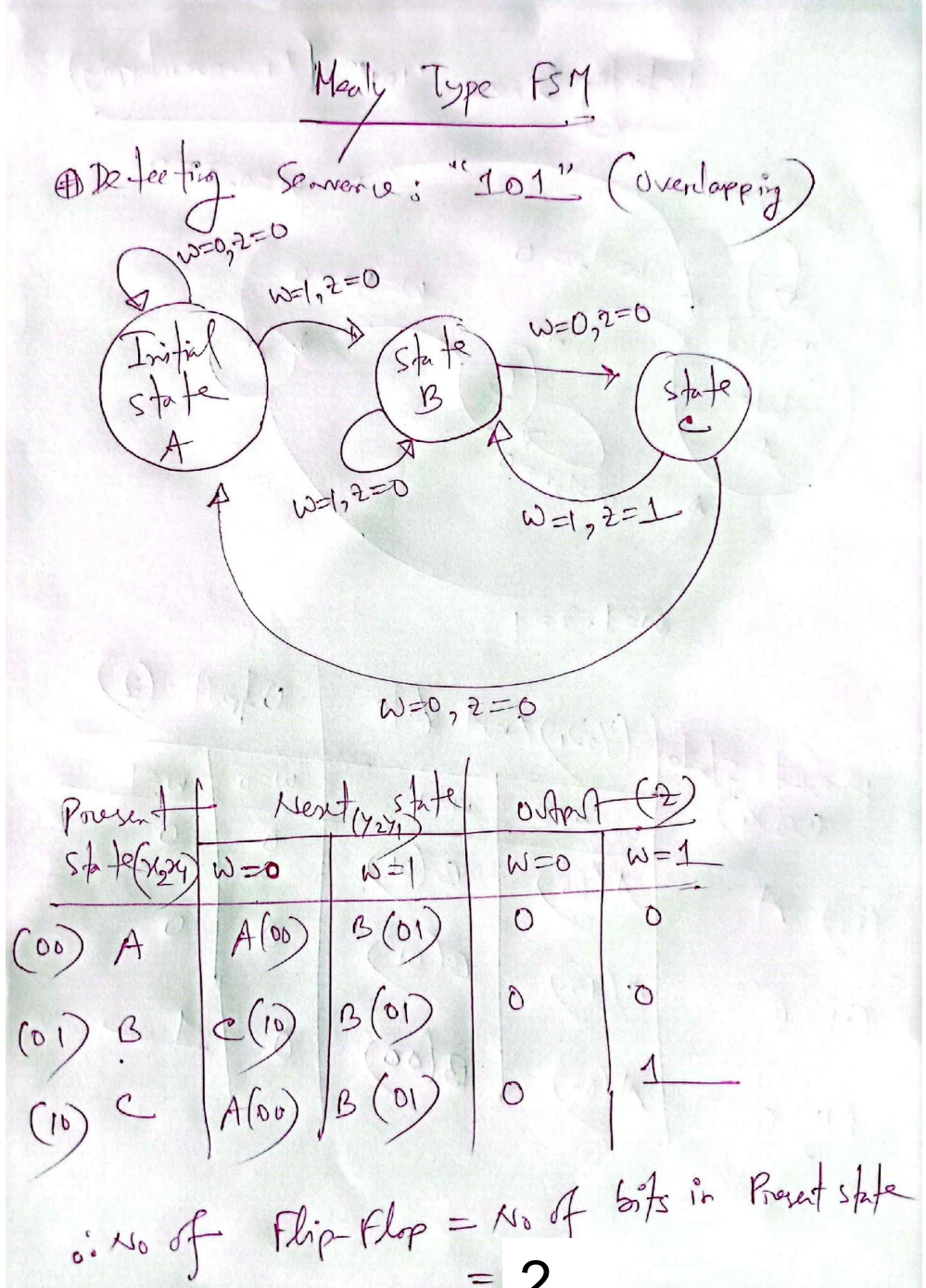
	Present state $y_3y_2y_1$	Next state $w = 0$ $Y_3Y_2Y_1$	Next state $w = 1$ $Y_3Y_2Y_1$	Output z
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

Schemes->

- Binary encoding
- Gray encoding
- **One-hot encoding**

How many flipflops are required? **Ans: 3**

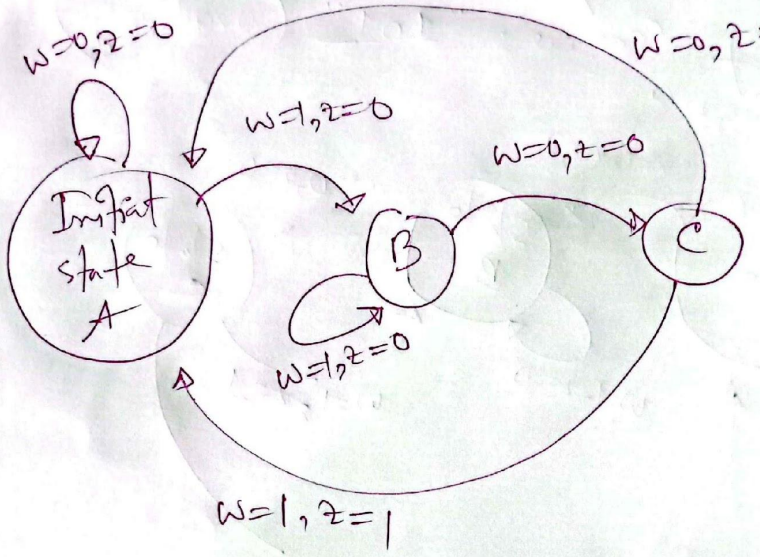
Example 2: A Simple Input Pattern ('101' Overlapping Sequence) Detection Circuit (Mealy Type)



Important:

In Mealy Type FSM, you do not need to declare a separate state for your final output (for this example: no separate state for '101' is required). After getting your output z = 1, you will start your 2nd loop (Start finding z=1 for the 2nd time).

Detecting Sequence "101" (Non-overlapping)



Present state (x_2x_1)	Next state (Y_2Y_1)		Output (z)	
	w=0	w=1	w=0	w=1
(00) A	A(00)	B(01)	0	0
(01) B	C(10)	B(01)	0	0
(10) C	A(00)	A (00)	0	1

Example 2: A Simple Input Pattern ('101' Non-Overlapping Sequence) Detection Circuit (Mealy Type)

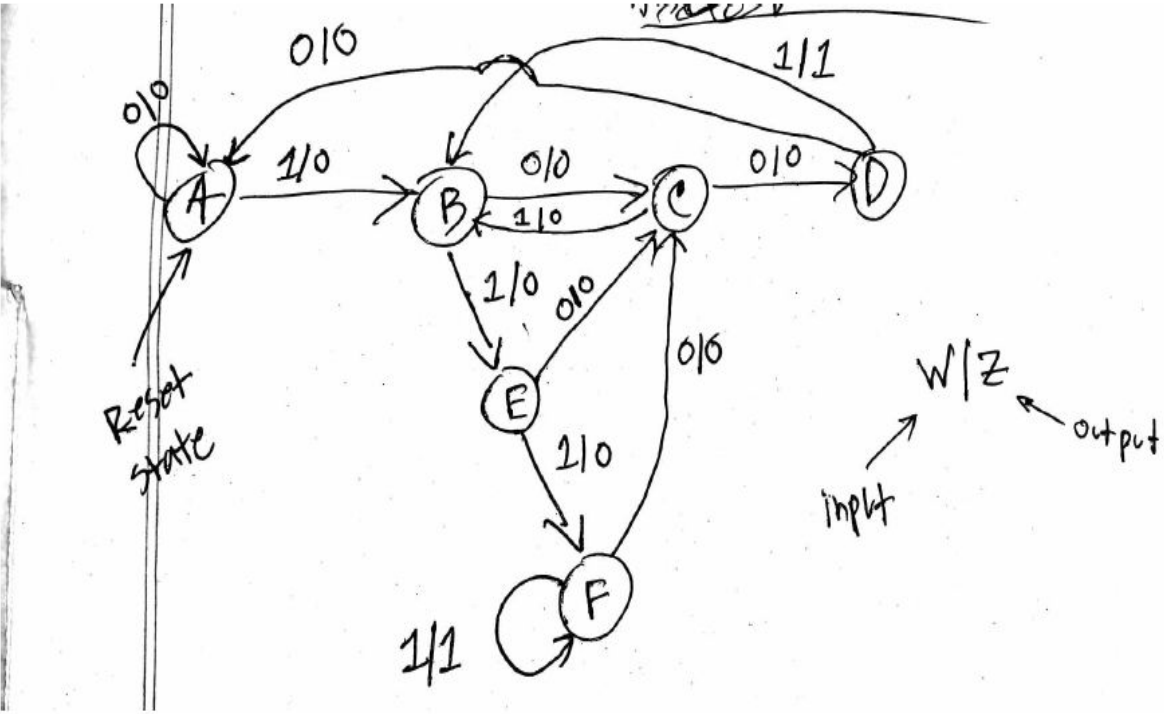
Example 3:

1.	1. A finite state machine that has to generate z = 1 when the previous four values of input w were 1001 or 1111; otherwise, z = 0. Overlapping input patters are allowed. The machine also has a negative reset. An example of the desired behavior is w : 0101 1110 0110 0111 11 z : 0000 0010 0100 0100 11	
(a)	**Design** the state diagram. Mark your reset states and transitions clearly. Is this machine Mealy type or Moore type?	**3**
(b)	**Derive** the assigned table and write the Verilog code to implement the following FSM.	**4**
(c)	**Validate** your answer with the appropriate input and output waveforms on a timing diagram. Include all possible combinations of the input (w).	**3**

Solution:

The Machine is Mealy type
My Reset state is A State
W = Input Sequence
Z = Output

Format is W/Z



Present State	Next State		Output	
	W=0	W=1	W=0	W=1
A	A	B	0	0
B	C	E	0	0
C	D	B	0	0
D	A	B	0	1
E	C	F	0	0
F	C	F	0	1

State Assigned Table:

A = ~~000~~
B = 010
C = 0

A = 000
B = 001
C = 010
D = 011
E = 100
F = 101

Present state	Next State		Out put	
	W=0	W=1	W=0	W=1
000	000	001	0	0
001	010	100	0	0
010	011	001	0	0
011	000	001	0	1
100	010	101	0	0
101	010	101	0	1

Example 4:

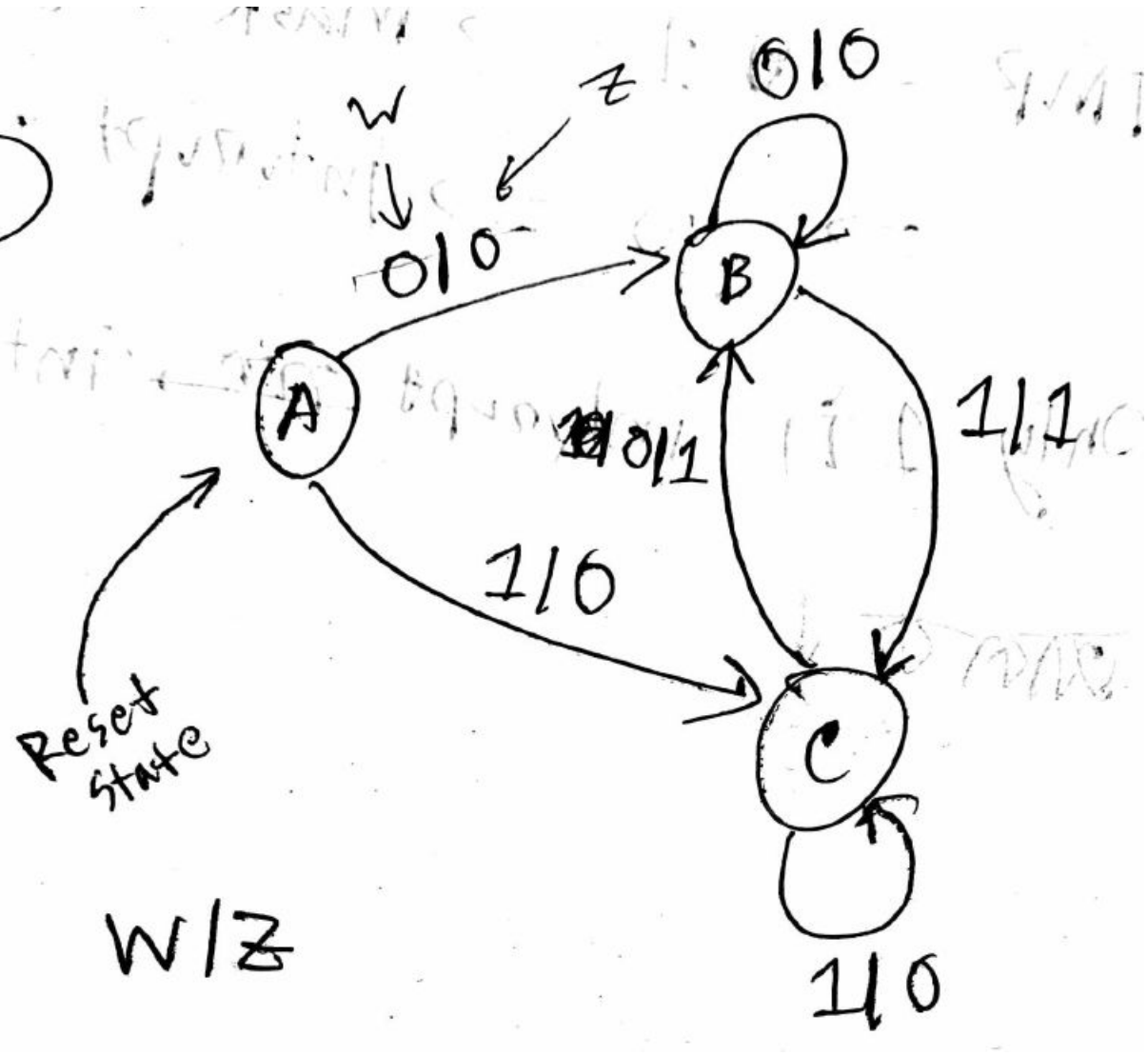
The input-output behavior of a sequence detector FSM is given below.

CLK	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}	t_{11}	t_{12}	t_{13}	t_{14}	t_{15}	t_{16}	t_{17}	t_{18}
w	**1**	**0**	**0**	**1**	1	**1**	**0**	0	0	0	0	0	0	**0**	**1**	**0**	0	0
z	0	**1**	0	**1**	0	0	**1**	0	0	0	0	0	0	0	**1**	**1**	0	0

(a)	**Identify** the sequence and the type of FSM for the above input-output combinations. [Hint: The above table contains two sequences]	**1+1**
(b)	**Design** the state diagram, the state table, and the ***gray-encoded*** state assigned table. **[Overlapping allowed].**	**3+2+2**
(c)	**Determine** the logic expressions of the next state and output variables using k-maps.	**6**

**** Alternating 0 and 1**
**** Two consecutive inputs are different**

Solution:

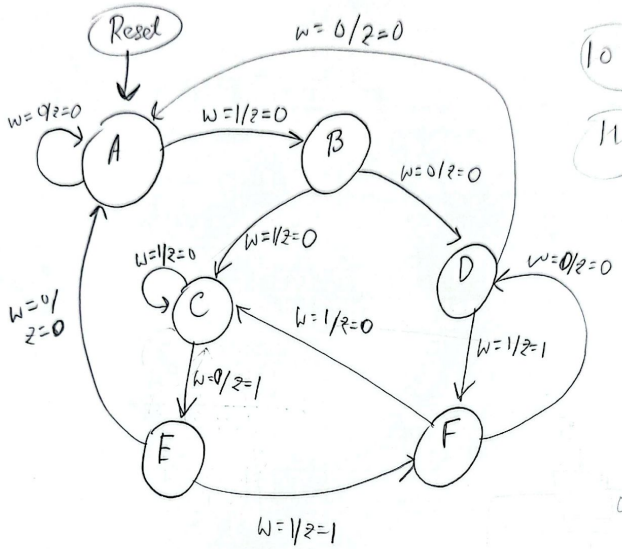


Present State	Next State		Output Z	
	W=0	W=1	W=0	W=1
A	B	C	0	0
B	B	C	0	1
C	B	C	1	0

Present state	Next State		Output Z	
	W=0	W=1	W=0	W=1
0 0	0 1	11	0	0
0 1	0 1	11	0	1
11	0 1	11	1	0

Draw the state diagram for detecting 110 or 101 overlapping sequence in a data stream. Output,z should be high in the same clock cycle. (b) Also derive the state assignment table for this. (c) Finally derive the output expression for this design.

Mealy type (110 or 101 overlapping)

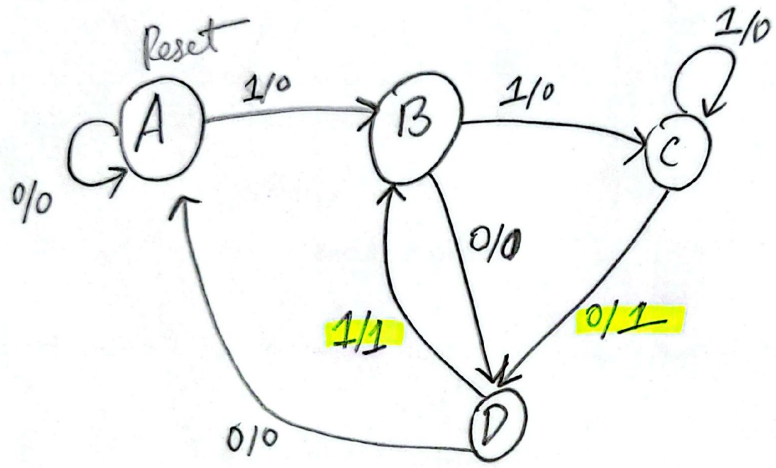


P.S $y_2y_1y_0$	N.S, $Y_2Y_1Y_0$ w=0	N.S, $Y_2Y_1Y_0$ w=1	z w=0	z w=1
000 (A)	000	001	0	0
001 (B)	011	010	0	0
010 (C)	100	010	1	0
011 (D)	000	101	0	1
100 (E)	000	101	0	1
101 (F)	011	010	0	0

z: y_2y_1 / y_0w	00	01	11	10
00	0	1	d	0
01	0	0	d	1
11	0	1	d	0
10	0	0	d	0

$$z = \bar{w}\bar{y_0}y_1 + w\bar{y_0}y_2 + wy_0y_1$$

(a)Draw the state diagram for detecting 110 or 101 overlapping sequence in a data stream. Output,z should be high in the next clock cycle. (b) Also derive the state assignment table for this. (c) Finally derive the output expression for this design.



P.S, y_2y_1	N.S, Y_2Y_1		Z	
	w=0	w=1	w=0	w=1
A(00)	A	B	0	0
B(01)	D	C	0	0
C(10)	D	C	1	0
D(11)	A	B	0	1

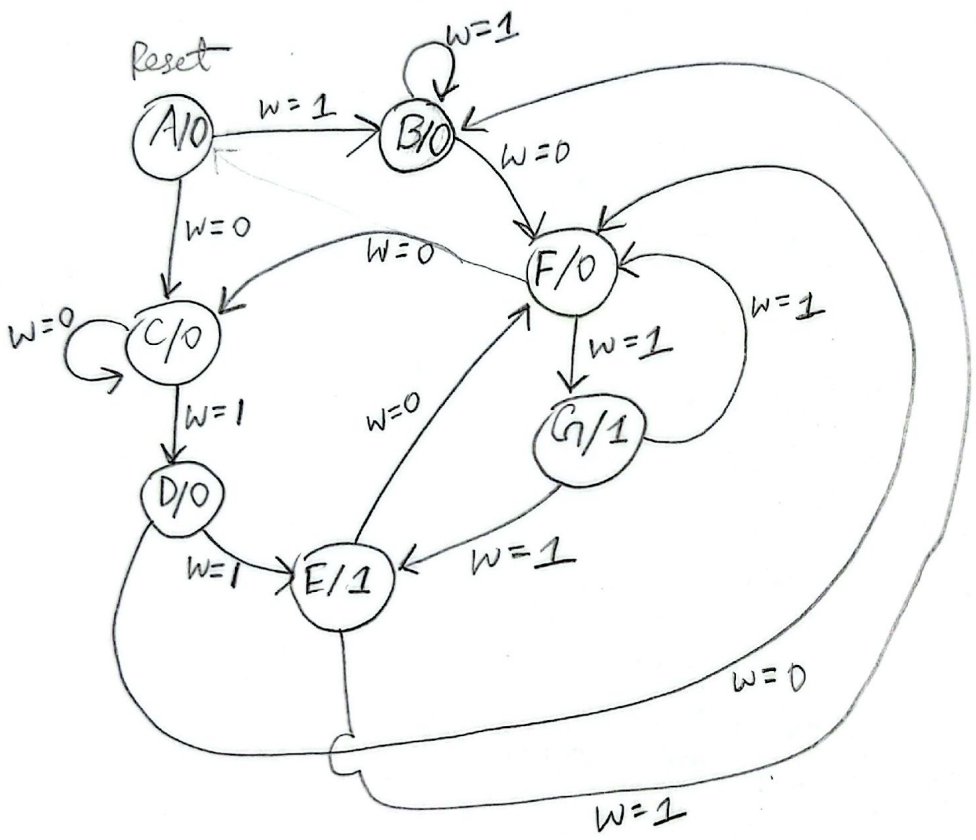
$$\rightarrow Z = \bar{w}y_2\bar{y}_1 + y_2y_1 \cdot w$$

$$= y_2(\bar{w}\bar{y}_1 + wy_1)$$

101 or 011 Overlapping Moore FSM

101 or 011 detector:

Set-A



10101

01101

10101

011011

1. (CO2) Drek and Simon are playing *Scale Wars*, a competitive game where each controls a powerful dragon—Drek's *Drogon* and Simon's *Rhaegal*. The goal is to level up their dragon by achieving consecutive wins. In each match, the outcomes are as follows: if Drek wins, a=1,b=0; if Simon wins, a=0,b=1; if both dragons collapse simultaneously, a=1,b=1; and if the game crashes suddenly (a rare event), a=0,b=0. If *Drogon* wins only once since it is a legendary dragon, it triggers z1=1 & levels up. However, *Rhaegal* triggers z2=1 and can level up only if it wins two consecutive times. Additionally, overlapping wins are **not counted** for *Rhaegal*. For example, if *Rhaegal* wins three consecutive rounds, only one level-up is triggered for the first two wins. Another level-up requires two additional consecutive wins. An example is shown in the table below:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	0	1	1	1	0	0	0	0	1	1	0	0	1	1	0
b	0	0	0	0	1	1	1	1	1	0	1	0	1	1	1
z1	0	0	**1**	**1**	**1**	0	0	0	0	0	**1**	0	0	0	0
z2	0	0	0	0	0	0	**1**	0	**1**	0	0	0	0	0	0

(a) Determine the type of FSM that would be needed for the given problem. **1**

(b) Design the state diagram for the corresponding FSM. Note that you will have to use a maximum of **four states** in the state diagram. There will be a deduction in marks if you take more states. **3**

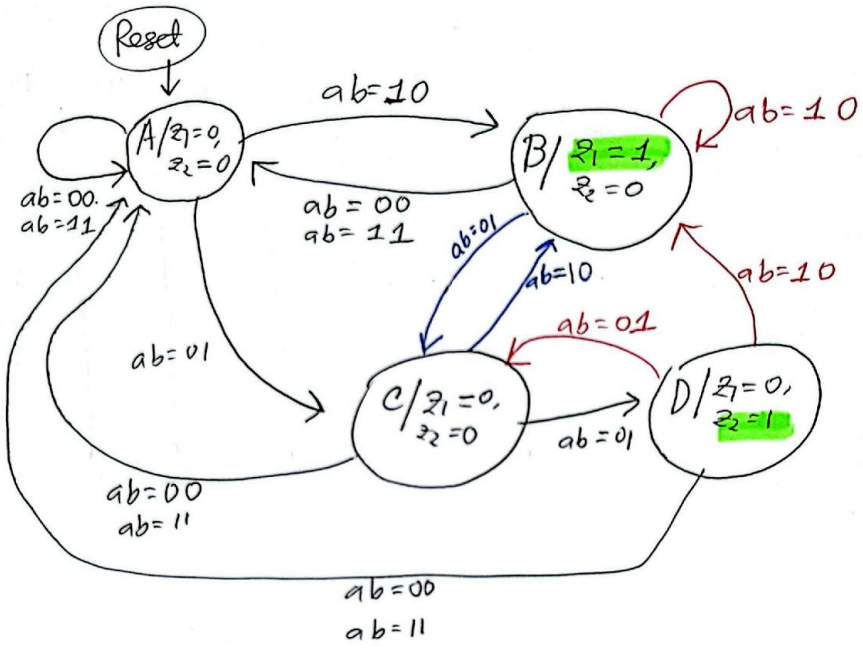
(c) Derive the state-assigned table using **Grey encoding** from your state diagram in (b). **3**

(d) Evaluate the output **z1** and **z2** for the FSM & calculate the number of flip-flops required for the circuit implementation. (You don't need to draw the circuit implementation).

3

a) Moore type (both sets)

b) State diagram:



(c) State Assignment table using gray coding

(A → 00, B → 01, C → 11, D → 10)

Present state $y_1 y_0$	Next state, $Y_1 Y_0$				outputs	
	ab= 00	01	10	11	z_1	z_2
00 (A)	00	11	01	00	0	0
01 (B)	00	11	01	00	1	0
11 (C)	00	10	01	00	0	0
10 (D)	00	11	01	00	0	1

(d)

$z_1 = \overline{y_1}\, y_0$

$z_2 = y_1 \overline{y_0}$

total required flipflops = 2